

* 해당 문제는 실습시간에 배운 예제 전체를 응용한 버전이다. 각 문제당 조건이 하나라도 빠지면 그 문제는 오답처리 되며 부분점수는 없습니다.

(1)

process_steps() 함수는 특정 조건을 만족하지 않으면 false를 반환한다. 이 함수가 true를 반환하기 위해서 반드시 충족해야 하는 조건 3가지를 적고, 각 조건이 어느 라인에서 체크가 되는지 정확한 근거를 들어 설명하시오.

(2)

제공된 예제에 새로운 기능을 추가하려고 한다. 특정 유저의 최근 로그 5개만 출력하는 show_last_logs(uid) 함수를 작성하라. 단, 로그는 data_store["logs"]에 존재하며, 유저 관련 로그만 필터링해야 한다.

(3)

문자열 분석 기능을 확장한다. count_words() 함수는 공백만 기준으로 단어를 나누고 있다. 다음 조건을 만족하도록 코드를 예제 형식에 맞게 추가하라 :

- ✓ 연속된 공백이 있어도 정확히 단어 수를 계산해야 한다.
- ✓ . , ! ? 등이 등장해도 단어가 끊이지 않도록 해야 한다.

(4)

예제의 정렬 함수 sort_users_by_score()는 내림차순 정렬이다. 이를 유저 이름 기준 오름차순 정렬 함수로 바꾸어 sort_users_by_name()을 작성하라.

(5)

encrypt_text()는 단순한 Shift 암호로 쓰인다. 다음 조건을 모두 만족하도록 encrypt_text(text, shift)를 추가하라.

- ✓ a-z, A-Z 범위만 shift 적용
- ✓ 나머지 공백, 숫자, 특수문자는 그대로 유지
- ✓ Z까지 밀리면 다시 A로 순환되도록 처리

(6)

주어진 예제의 recursive_reverse()를 참고하여 문자열의 모음만 뒤집어서 반환하는 recursive_reverse(text) 함수를 작성하여라. (ex: Hello → Holle)

(7)

예제에서 Task는 steps list를 가진다. 이 구조를 확장하여 아래의 조건에 따라 Task 마다 난이도 값을 추가하라.

- ✓ create_task()에 difficulty(int) 추가
- ✓ 난이도 x steps 개수만큼 점수 추가 지급
- ✓ 난이도 점수는 상(3점), 중(2점), 하(1점)

(8)

simulate_points()는 uid %3을 이용하여 점수를 계산한다. 다음 규칙으로 점수를 재정의하도록 함수를 수정하라.

- ✓ uid가 짝수면 매 라운드 +2점
- ✓ uid가 홀수면 매 라운드 -1점
- ✓ 단, 총점이 0 아래로 떨어지지 않도록 조건처리

(9)

예제는 mode를 10까지 정의되어있다. 여기에 “11. 특정 단어를 포함하는 로그만 출력”하는 기능을 추가하라. 단어가 입력되면, 해당 단어가 포함된 로그 문자열만 출력되어야 한다.

(10)

예제는 summarize_users()를 제공하고 있다. 여기에 다음 정보를 포함하도록 summarize_users()를 다시 작성하라.

- ✓ 전체 유저 수
- ✓ 전체 평균 점수
- ✓ 최고 점수 유저 uid
- ✓ 최저 점수 유저 uid
- ✓ 이름을 알파벳 순으로 정렬한 list

<pre>data_store = { "users": [], "tasks": [], "logs": [] } def add_user(uid, name): user = { "uid": uid, "name": name, "score": 0, "history": [] } data_store["users"].append(user) return user def find_user(uid): for u in data_store["users"]: if u["uid"] == uid: return u return None def update_score(uid, value): u = find_user(uid) if u: u["score"] += value u["history"].append(value) return True return False def add_log(message): data_store["logs"].append(message) def create_task(tid, title, steps): task = { "tid": tid, "title": title, "steps": steps, "completed_by": [] } data_store["tasks"].append(task) return task def get_task(tid): for t in data_store["tasks"]: if t["tid"] == tid: return t return None def complete_task(uid, tid): user = find_user(uid) task = get_task(tid) if user and task: if uid not in task["completed_by"]: task["completed_by"].append(uid) update_score(uid, len(task["steps"])) add_log("User " + str(uid) + " completed task " + str(tid)) return True return False def verify_step(step_text, requirement): matched = 0 for c in step_text:</pre>	<pre> if c in requirement: matched += 1 return matched == len(step_text) def process_steps(uid, tid, provided_steps): task = get_task(tid) if not task: return False required = task["steps"] if len(required) != len(provided_steps): return False for i in range(len(required)): ok = verify_step(provided_steps[i], required[i]) if not ok: return False return complete_task(uid, tid) def summarize_user(uid): u = find_user(uid) if not u: return None total = 0 count = 0 for h in u["history"]: total += h count += 1 avg = total / count if count > 0 else 0 return { "uid": uid, "name": u["name"], "score": u["score"], "task_count": count, "average": avg } def count_words(text): word = "" words = [] for c in text: if c == " ": if word != "": words.append(word) word = "" else: word += c if word != "": words.append(word) return len(words) def find_longest_word(text): longest = "" word = "" for c in text: if c == " ": if len(word) > len(longest): longest = word word = "" else: word += c</pre>	<pre> if len(word) > len(longest): longest = word return longest def search_users_by_letter(letter): result = [] for u in data_store["users"]: if letter in u["name"]: result.append(u) return result def sort_users_by_score(): users = data_store["users"] n = len(users) for i in range(n): for j in range(0, n - i - 1): if users[j]["score"] < users[j+1]["score"]: temp = users[j] users[j] = users[j+1] users[j+1] = temp return users def encrypt_text(text, shift): result = "" for c in text: code = ord(c) code += shift result += chr(code) return result def decrypt_text(text, shift): result = "" for c in text: code = ord(c) code -= shift result += chr(code) return result def recursive_sum(n): if n <= 0: return 0 return n + recursive_sum(n - 1) def recursive_reverse(text): if text == "": return "" return recursive_reverse(text[1:]) + text[0] def simulate_points(uid, rounds): total = 0 for r in range(rounds): gained = (r % 5) + uid % 3 total += gained update_score(uid, gained) return total def initialize_sample_data(): add_user(1, "Alice") add_user(2, "Bob") add_user(3, "Charlie") create_task(101, "Basic Task", ["abc", "xy", "opq"]) create_task(102, "Name Task", ["aa", "bb", "cc"])</pre>	<pre>initialize_sample_data() def run(): running = True while running: mode = input() if mode == "1": uid = int(input()) name = input() add_user(uid, name) elif mode == "2": uid = int(input()) tid = int(input()) step_count = int(input()) steps = [] for i in range(step_count): steps.append(input()) process_steps(uid, tid, steps) elif mode == "3": uid = int(input()) res = summarize_user(uid) if res: print(res["uid"]) print(res["name"]) print(res["score"]) print(res["average"]) elif mode == "4": txt = input() print(count_words(txt)) print(find_longest_word(txt)) elif mode == "5": letter = input() result = search_users_by_letter(letter) for u in result: print(u["uid"], u["name"]) elif mode == "6": sorted_users = sort_users_by_score() for u in sorted_users: print(u["uid"], u["score"]) elif mode == "7": txt = input() shift = int(input()) print(encrypt_text(txt, shift)) elif mode == "8": txt = input() shift = int(input()) print(decrypt_text(txt, shift)) elif mode == "9": uid = int(input()) rounds = int(input()) print(simulate_points(uid, rounds)) elif mode == "10": n = int(input()) print(recursive_sum(n)) t = input() print(recursive_reverse(t)) elif mode == "0": running = False run()</pre>
--	--	---	---